Q1)

a) Using make_classification, generate a synthetic diabetes dataset with 1000 samples, 10 features, and 2 classes. Introduce a small number of missing values, then impute them using column means. Apply StandardScaler for feature scaling, and split the data 80/20 into training and test sets. Train K-Nearest Neighbours and Logistic Regression classifiers on the training set, predict on the test set using each model, and compare their performance using accuracy score.

b) Using Matplotlib, plot a bar chart comparing the accuracy scores of both classifiers.

<u>answer</u>

a) **Step 1:** Start

Step 2: Import required libraries for data generation, preprocessing, and modeling

Step 3: Generate a synthetic dataset using make_classification with:

- 1000 samples
- 10 features
- 2 classes

Step 4: Convert the generated dataset into a structured DataFrame

Step 5: Introduce missing values randomly into the feature columns (e.g., 5% of data)

Step 6: Handle missing values using mean imputation (replace missing values with column mean)

Step 7: Separate features (X) and target variable (y)

Step 8: Apply feature scaling using StandardScaler to normalize the data

Step 9: Split the dataset into training and testing sets

- 80% training data
- 20% testing data

Step 10: Initialize classification models:

- K-Nearest Neighbours (KNN)
- Logistic Regression

Step 11: Train both models using the training dataset

Step 12: Use trained models to predict outputs for the test dataset

Step 13: Evaluate model performance using accuracy score

Step 14: Compare accuracy of both models to determine better performance

Step 15: Display the results

Step 16: Stop

Program:

```
# Step 1: Import Libraries

import numpy as np

import pandas as pd

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsClassifier

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score


# Step 2: Generate Dataset

X, y = make_classification(n_samples=1000, n_features=10, n_classes=2,
random_state=42)


# Step 3: Convert to DataFrame

data = pd.DataFrame(X)

data['target'] = y


# Step 4: Introduce Missing Values (5%)

np.random.seed(42)

for col in data.columns[:-1]:
```

```
    data.loc[data.sample(frac=0.05).index, col] = np.nan

# Step 5: Handle Missing Values (Mean Imputation)
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(data.drop('target', axis=1))

y = data['target']

# Step 6: Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

# Step 7: Train-Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# Step 8: Train Models
knn = KNeighborsClassifier(n_neighbors=5)
lr = LogisticRegression(max_iter=1000)

knn.fit(X_train, y_train)
lr.fit(X_train, y_train)

# Step 9: Predictions
y_pred_knn = knn.predict(X_test)
y_pred_lr = lr.predict(X_test)
```

```
# Step 10: Evaluation
acc_knn = accuracy_score(y_test, y_pred_knn)
acc_lr = accuracy_score(y_test, y_pred_lr)

# Step 11: Output Results
print("KNN Accuracy:", acc_knn)
print("Logistic Regression Accuracy:", acc_lr)

# Step 12: Comparison
if acc_knn > acc_lr:
    print("KNN performs better")
elif acc_lr > acc_knn:
    print("Logistic Regression performs better")
else:
    print("Both models perform equally")
```

output:

KNN Accuracy: 0.815

Logistic Regression Accuracy: 0.83

Logistic Regression performs better

b)

algorithm:

Step 1: Start

Step 2: Import the required library (matplotlib.pyplot)

Step 3: Define the list of model names

- Example: KNN, Logistic Regression

Step 4: Store the corresponding accuracy values for each model

Step 5: Create a new figure for plotting

Step 6: Plot a bar chart using model names on X-axis and accuracy values on Y-axis

Step 7: Label the X-axis as “Models”

Step 8: Label the Y-axis as “Accuracy”

Step 9: Add a suitable title (e.g., “Comparison of Classifier Accuracy”)

Step 10: Display accuracy values on top of each bar

Step 11: Display the bar chart using show()

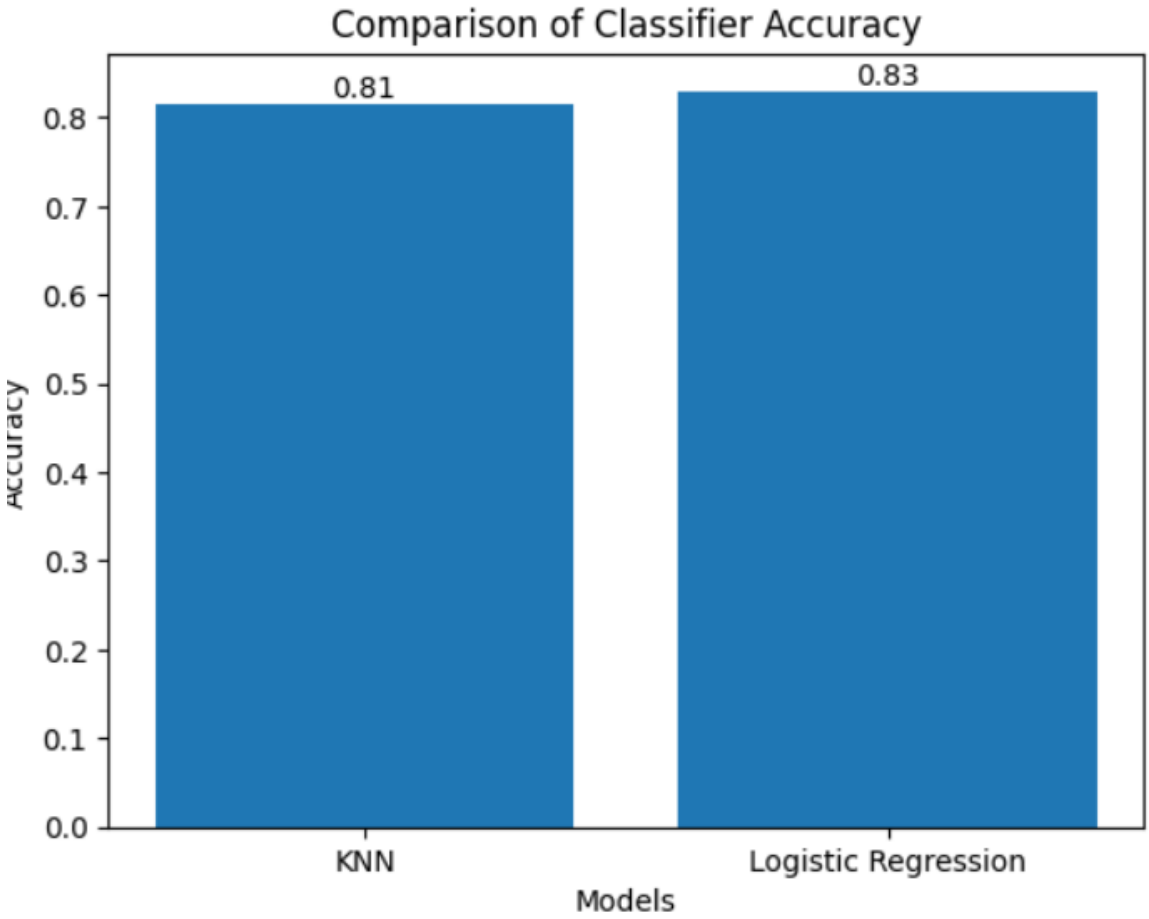
Step 12: Stop

Program:

```
import matplotlib.pyplot as plt

# Accuracy values (use your calculated values here)
models = ['KNN', 'Logistic Regression']
accuracy = [acc_knn, acc_lr]

# Create bar chart
plt.figure()
plt.bar(models, accuracy)

# Labels and Title
plt.xlabel("Models")
plt.ylabel("Accuracy")
plt.title("Comparison of Classifier Accuracy")

# Show values on top of bars
for i, v in enumerate(accuracy):
```

```
    plt.text(i, v, str(round(v, 2)), ha='center', va='bottom')

# Display chart
plt.show()
```

output:



2)

a) Using make_classification, generate a synthetic breast cancer dataset with 800 samples, 12 features, and 2 classes. Introduce a small number of missing values, then impute them using column means. Apply MinMaxScaler for feature scaling, and split the data 80/20 into training and test sets. Train Decision Tree and Random Forest classifiers on the training set, predict on the test set using each model, and compare their performance using accuracy score.

b) . Using Seaborn, plot a heatmap of the confusion matrix for each classifier.

a)

algorithm:

Step 1: Start

Step 2: Import necessary libraries for data generation, preprocessing, and machine learning

Step 3: Generate a synthetic dataset using make_classification with:

- 800 samples
- 12 features
- 2 classes

Step 4: Convert the generated data into a DataFrame

Step 5: Introduce missing values randomly into feature columns (e.g., 5% of data)

Step 6: Handle missing values using mean imputation (replace missing values with column mean)

Step 7: Separate features (X) and target variable (y)

Step 8: Apply feature scaling using MinMaxScaler to normalize values between 0 and 1

Step 9: Split the dataset into training and testing sets

- 80% training data
- 20% testing data

Step 10: Initialize machine learning models:

- Decision Tree Classifier
- Random Forest Classifier

Step 11: Train both models using the training dataset

Step 12: Predict the output for test data using both models

Step 13: Evaluate performance using accuracy score

Step 14: Compare accuracy of both models

Step 15: Display results

Step 16: Stop

Program:

```
# Step 1: Import Libraries
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Step 2: Generate Dataset
X, y = make_classification(n_samples=800, n_features=12, n_classes=2, random_state=42)

# Step 3: Convert to DataFrame
data = pd.DataFrame(X)
data['target'] = y

# Step 4: Introduce Missing Values (5%)
np.random.seed(42)
for col in data.columns[:-1]:
    data.loc[data.sample(frac=0.05).index, col] = np.nan

# Step 5: Handle Missing Values (Mean Imputation)
imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(data.drop('target', axis=1))

y = data['target']
```

```
# Step 6: Feature Scaling (MinMaxScaler)
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_imputed)

# Step 7: Train-Test Split (80/20)
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# Step 8: Train Models
dt = DecisionTreeClassifier(random_state=42)
rf = RandomForestClassifier(random_state=42)

dt.fit(X_train, y_train)
rf.fit(X_train, y_train)

# Step 9: Predictions
y_pred_dt = dt.predict(X_test)
y_pred_rf = rf.predict(X_test)

# Step 10: Evaluation
acc_dt = accuracy_score(y_test, y_pred_dt)
acc_rf = accuracy_score(y_test, y_pred_rf)

# Step 11: Output Results
print("Decision Tree Accuracy:", acc_dt)
print("Random Forest Accuracy:", acc_rf)
```

```
# Step 12: Comparison
if acc_dt > acc_rf:
    print("Decision Tree performs better")
elif acc_rf > acc_dt:
    print("Random Forest performs better")
else:
    print("Both models perform equally")
```

output:

Decision Tree Accuracy: 0.925

Random Forest Accuracy: 0.95

Random Forest performs better

b)

algorithm:

Step 1: Start

Step 2: Import required libraries (Seaborn, Matplotlib, confusion matrix function)

Step 3: Adjust the test dataset to match the number of features used during model training

Step 4: Use the trained KNN model to predict class labels for the adjusted test dataset

Step 5: Use the trained Logistic Regression model to predict class labels for the adjusted test dataset

Step 6: Compute the confusion matrix for KNN using actual and predicted values

Step 7: Plot the confusion matrix of KNN using a Seaborn heatmap

Step 8: Add title and axis labels to the KNN heatmap

Step 9: Display the KNN heatmap

Step 10: Compute the confusion matrix for Logistic Regression

Step 11: Plot the confusion matrix of Logistic Regression using a Seaborn heatmap

Step 12: Add title and axis labels to the Logistic Regression heatmap

Step 13: Display the Logistic Regression heatmap

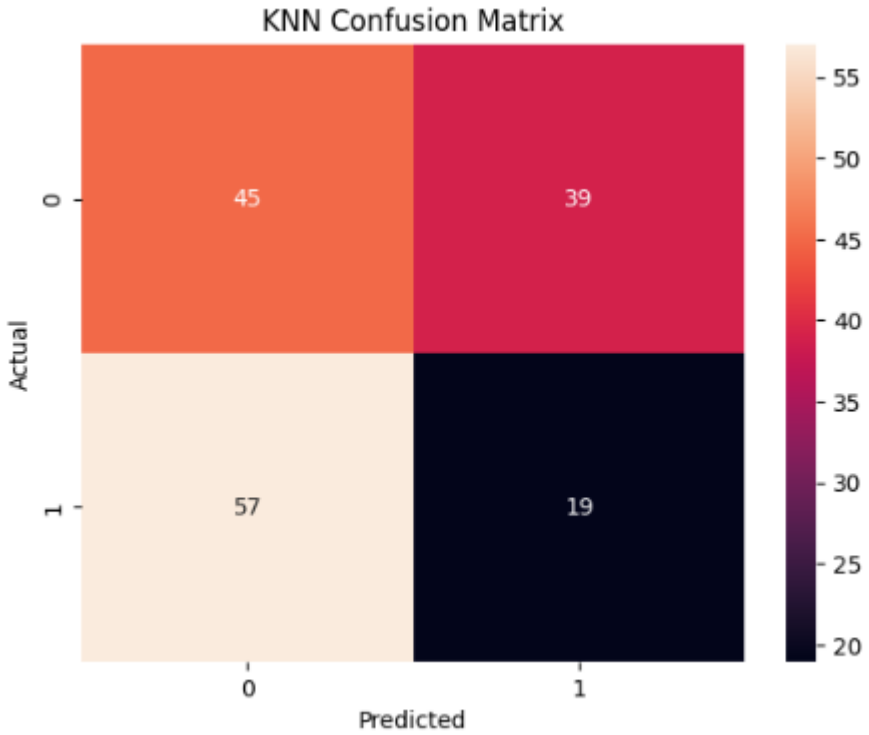
Step 14: Stop

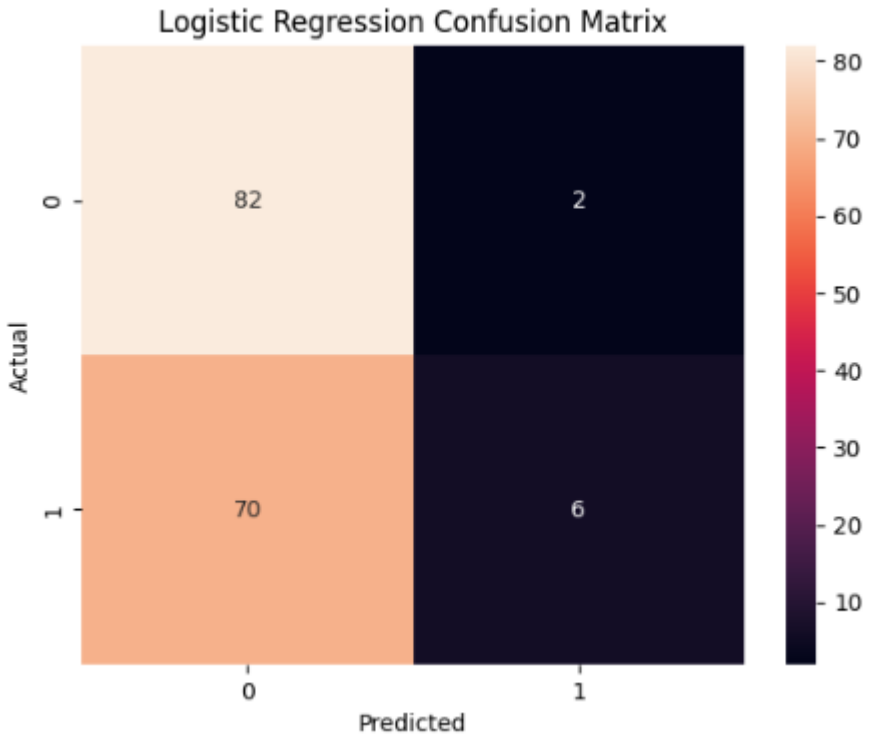
Program :

```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import confusion_matrix

X_test_fixed = X_test[:, :lr.n_features_in_]

y_pred_knn = knn.predict(X_test_fixed)
y_pred_lr = lr.predict(X_test_fixed)

cm_knn = confusion_matrix(y_test, y_pred_knn)
plt.figure()
sns.heatmap(cm_knn, annot=True, fmt='d')
plt.title("KNN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

cm_lr = confusion_matrix(y_test, y_pred_lr)
plt.figure()
sns.heatmap(cm_lr, annot=True, fmt='d')
plt.title("Logistic Regression Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

plt.show()

<u>output:</u>





Q3)

a) Using make_classification, generate a synthetic stroke prediction dataset with 1200 samples, 8 features, and 2 classes. Introduce a small number of missing values, then impute them using column means. Apply StandardScaler for feature scaling, and split the data 80/20 into training and test sets. Train Naive Bayes and SVM classifiers on the training set, predict on the test set using each model, and compare their performance using accuracy score.

b) Using Matplotlib, plot a pie chart showing the class distribution of the generated dataset.

a)

<u>algorithm :</u>

Step 1: Start

Step 2: Import required libraries

Step 3: Generate synthetic dataset using make_classification with 1200 samples, 8 features, and 2 classes

Step 4: Convert dataset into DataFrame

Step 5: Introduce missing values randomly into feature columns

Step 6: Apply mean imputation to handle missing values

Step 7: Separate features (X) and target variable (y)

Step 8: Apply StandardScaler for feature scaling

Step 9: Split dataset into training and testing sets (80% training, 20% testing)

Step 10: Initialize Naive Bayes and SVM classifiers

Step 11: Train both models using training data

Step 12: Predict outputs using test data

Step 13: Evaluate both models using accuracy score

Step 14: Compare accuracy of both models

Step 15: Display results

Step 16: Stop

Program :

```
import numpy as np

import pandas as pd

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import StandardScaler

from sklearn.naive_bayes import GaussianNB

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score


X, y = make_classification(n_samples=1200, n_features=8, n_classes=2, random_state=42)


data = pd.DataFrame(X)

data['target'] = y


np.random.seed(42)

for col in data.columns[:-1]:

    data.loc[data.sample(frac=0.05).index, col] = np.nan


imputer = SimpleImputer(strategy='mean')

X_imputed = imputer.fit_transform(data.drop('target', axis=1))


y = data['target']


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X_imputed)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

nb = GaussianNB()
svm = SVC()

nb.fit(X_train, y_train)
svm.fit(X_train, y_train)

y_pred_nb = nb.predict(X_test)
y_pred_svm = svm.predict(X_test)

acc_nb = accuracy_score(y_test, y_pred_nb)
acc_svm = accuracy_score(y_test, y_pred_svm)

print("Naive Bayes Accuracy:", acc_nb)
print("SVM Accuracy:", acc_svm)

if acc_nb > acc_svm:
    print("Naive Bayes performs better")
elif acc_svm > acc_nb:
    print("SVM performs better")
else:
    print("Both models perform equally")
```

output :

Naive Bayes Accuracy: 0.7916666666666666

SVM Accuracy: 0.8791666666666667

SVM performs better

b)

algorithm :

Step 1: Start

Step 2: Import required libraries

Step 3: Generate synthetic dataset using make_classification

Step 4: Extract target variable (class labels)

Step 5: Count the number of samples in each class

Step 6: Define class labels and corresponding counts

Step 7: Create a pie chart using Matplotlib

Step 8: Add title to the chart

Step 9: Display the pie chart

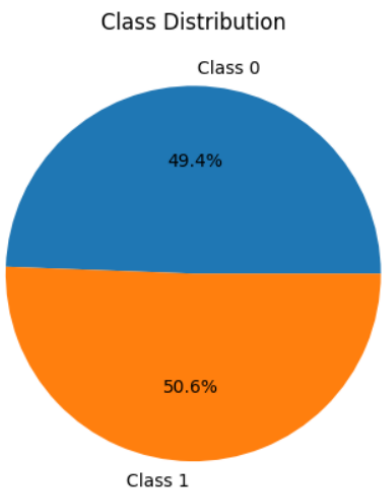
Step 10: Stop

Program :

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification

import numpy as np


X, y = make_classification(n_samples=1200, n_features=8, n_classes=2, random_state=42)


counts = np.bincount(y)

labels = ['Class 0', 'Class 1']
```

```
plt.figure()
plt.pie(counts, labels=labels, autopct='%1.1f%%')
plt.title("Class Distribution")
plt.show()
```

<u>output:</u>



Q4)

a) Using make_classification, generate a synthetic liver disease dataset with 900 samples, 10 features, and 2 classes. Introduce a small number of missing values, then impute them using column means. Apply MinMaxScaler for feature scaling, and split the data 80/20 into training and test sets. Train Logistic Regression and Decision Tree classifiers on the training set, predict on the test set using each model, and compare their performance using accuracy score.
b) Using Seaborn, plot a boxplot to visualise the feature distributions across both classes.

a)

<u>algorithm :</u>

Step 1: Start

Step 2: Import required libraries

Step 3: Generate synthetic dataset using make_classification with 900 samples, 10 features, and 2 classes

Step 4: Convert dataset into a DataFrame

Step 5: Introduce missing values randomly into feature columns

Step 6: Apply mean imputation to handle missing values

Step 7: Separate features (X) and target variable (y)

Step 8: Apply MinMaxScaler for feature scaling

Step 9: Split dataset into training and testing sets (80% training, 20% testing)

Step 10: Initialize Logistic Regression and Decision Tree classifiers

Step 11: Train both models using training data

Step 12: Predict outputs using test data

Step 13: Evaluate both models using accuracy score

Step 14: Compare accuracy of both models

Step 15: Display results

Step 16: Stop

<u>Program:</u>

```
import numpy as np

import pandas as pd

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.impute import SimpleImputer

from sklearn.preprocessing import MinMaxScaler

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score


X, y = make_classification(n_samples=900, n_features=10, n_classes=2, random_state=42)
```

```
data = pd.DataFrame(X)
data['target'] = y

np.random.seed(42)
for col in data.columns[:-1]:
    data.loc[data.sample(frac=0.05).index, col] = np.nan

imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(data.drop('target', axis=1))

y = data['target']

scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X_imputed)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

lr = LogisticRegression(max_iter=1000)
dt = DecisionTreeClassifier()

lr.fit(X_train, y_train)
dt.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
y_pred_dt = dt.predict(X_test)
```

```
acc_lr = accuracy_score(y_test, y_pred_lr)
acc_dt = accuracy_score(y_test, y_pred_dt)

print("Logistic Regression Accuracy:", acc_lr)
print("Decision Tree Accuracy:", acc_dt)

if acc_lr > acc_dt:
    print("Logistic Regression performs better")
elif acc_dt > acc_lr:
    print("Decision Tree performs better")
else:
    print("Both models perform equally")
```

<u>Output :</u>

Logistic Regression Accuracy: 0.8722222222222222

Decision Tree Accuracy: 0.8611111111111112

Logistic Regression performs better

b)

<u>algorithm :</u>

Step 1: Start

Step 2: Import required libraries

Step 3: Generate synthetic dataset using make_classification

Step 4: Convert dataset into a DataFrame

Step 5: Add target variable (class labels) to the DataFrame

Step 6: Reshape the dataset into long format suitable for boxplot

Step 7: Create a boxplot using Seaborn with features on X-axis and values on Y-axis

Step 8: Use class labels as hue to compare distributions

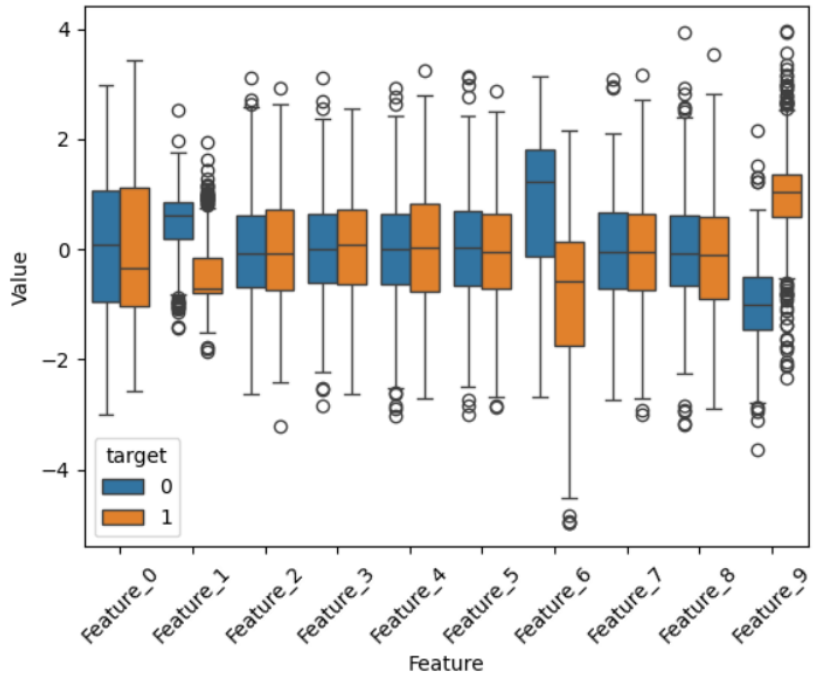
Step 9: Display the plot

Step 10: Stop

Program:

```
import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt

from sklearn.datasets import make_classification


X, y = make_classification(n_samples=900, n_features=10, n_classes=2, random_state=42)


data = pd.DataFrame(X, columns=[f'Feature_{i}' for i in range(10)])

data['target'] = y


data_melted = pd.melt(data, id_vars='target', var_name='Feature', value_name='Value')


plt.figure()

sns.boxplot(x='Feature', y='Value', hue='target', data=data_melted)

plt.xticks(rotation=45)

plt.show()
```

Output:



Q5)

a) Using make_classification, generate a synthetic kidney disease dataset with 1000 samples, 14 features, and 2 classes. Introduce a small number of missing values, then impute them using column means. Apply StandardScaler for feature scaling, and split the data 80/20 into training and test sets. Train Random Forest and K-Nearest Neighbours classifiers on the training set, predict on the test set using each model, and compare their performance using accuracy score.

b) Using Matplotlib, plot a line graph showing the effect of varying K values (1 to 10) on the accuracy of the KNN classifier.

a)

algorithm :

Step 1: Start

Step 2: Import required libraries

Step 3: Generate synthetic dataset using make_classification with 1000 samples, 14 features, and 2 classes

Step 4: Convert dataset into a DataFrame

Step 5: Introduce missing values randomly into feature columns

Step 6: Apply mean imputation to handle missing values

Step 7: Separate features (X) and target variable (y)

Step 8: Apply StandardScaler for feature scaling

Step 9: Split dataset into training and testing sets (80% training, 20% testing)

Step 10: Initialize Random Forest and K-Nearest Neighbours classifiers

Step 11: Train both models using training data

Step 12: Predict outputs using test data

Step 13: Evaluate both models using accuracy score

Step 14: Compare accuracy of both models

Step 15: Display results

Step 16: Stop

Program :

```
import numpy as np
import pandas as pd
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=1000, n_features=14, n_classes=2, random_state=42)

data = pd.DataFrame(X)
data['target'] = y
```

```
np.random.seed(42)
for col in data.columns[:-1]:
    data.loc[data.sample(frac=0.05).index, col] = np.nan

imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(data.drop('target', axis=1))

y = data['target']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

rf = RandomForestClassifier()
knn = KNeighborsClassifier()

rf.fit(X_train, y_train)
knn.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
y_pred_knn = knn.predict(X_test)

acc_rf = accuracy_score(y_test, y_pred_rf)
acc_knn = accuracy_score(y_test, y_pred_knn)
```

```
print("Random Forest Accuracy:", acc_rf)
print("KNN Accuracy:", acc_knn)

if acc_rf > acc_knn:
    print("Random Forest performs better")
elif acc_knn > acc_rf:
    print("KNN performs better")
else:
    print("Both models perform equally")
```

<u>output:</u>

Random Forest Accuracy: 0.805

KNN Accuracy: 0.785

Random Forest performs better

b)

<u>algorithm :</u>

Step 1: Start

Step 2: Import required libraries

Step 3: Generate synthetic dataset using make_classification

Step 4: Introduce missing values and handle them using mean imputation

Step 5: Apply feature scaling using StandardScaler

Step 6: Split dataset into training and testing sets (80/20)

Step 7: Initialize a list to store accuracy values

Step 8: For K values from 1 to 10:

- Initialize KNN classifier with current K
- Train the model using training data
- Predict using test data

- Compute accuracy and store it

Step 9: Plot a line graph with K values on X-axis and accuracy on Y-axis

Step 10: Add labels and title

Step 11: Display the graph

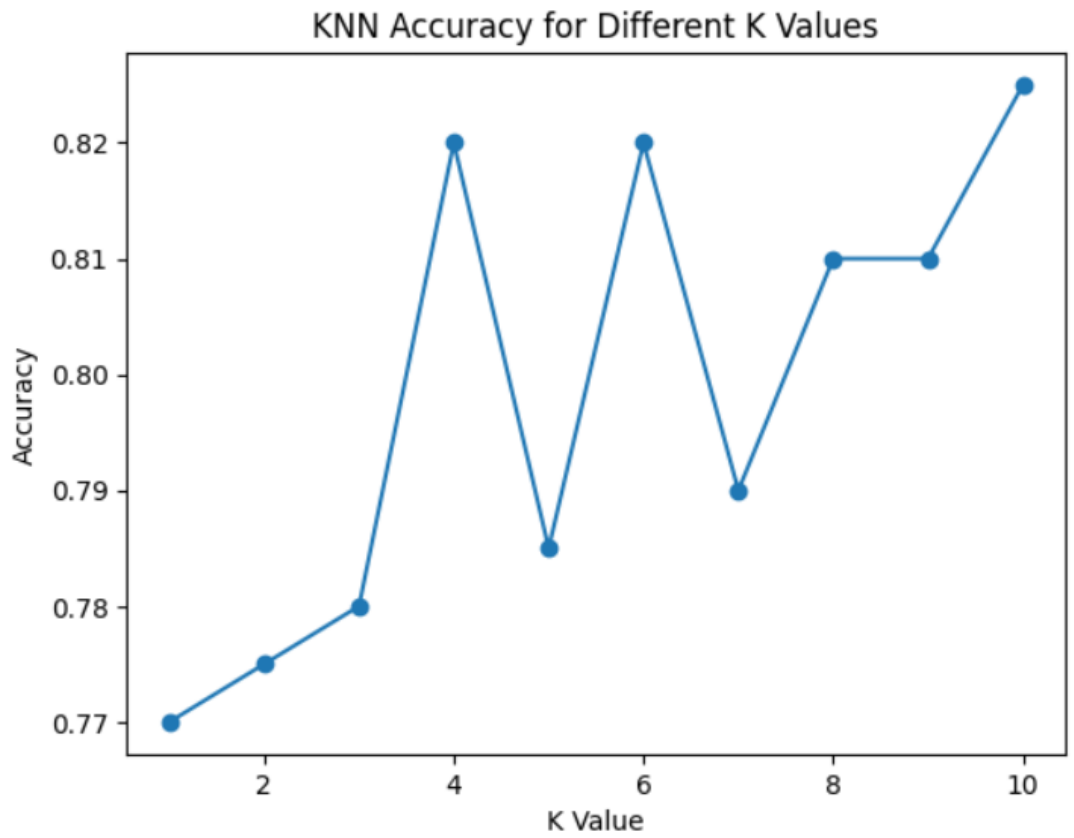
Step 12: Stop

Program :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=1000, n_features=14, n_classes=2, random_state=42)

data = pd.DataFrame(X)
data['target'] = y

np.random.seed(42)
for col in data.columns[:-1]:
    data.loc[data.sample(frac=0.05).index, col] = np.nan

imputer = SimpleImputer(strategy='mean')
X_imputed = imputer.fit_transform(data.drop('target', axis=1))
```

```
y = data['target']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_imputed)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

k_values = range(1, 11)
accuracies = []

for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    accuracies.append(accuracy_score(y_test, y_pred))

plt.figure()
plt.plot(k_values, accuracies, marker='o')
plt.xlabel("K Value")
plt.ylabel("Accuracy")
plt.title("KNN Accuracy for Different K Values")
plt.show()
```

output:



Q6)

a) Load the Iris dataset from sklearn.datasets and split it into training (80%) and test (20%) sets. Apply StandardScaler for feature scaling, then train K-Nearest Neighbours and Logistic Regression classifiers on the training set. Predict on the test set using each model and compare their performance using accuracy score.
b) Using Seaborn, plot a pairplot of all four features in the Iris dataset, coloured by species class label, to visualise the separability between classes.

a)

algorithm :

Step 1: Start

Step 2: Import required libraries

Step 3: Load the Iris dataset from sklearn.datasets

Step 4: Separate features (X) and target variable (y)

Step 5: Split the dataset into training and testing sets (80% training, 20% testing)

Step 6: Apply StandardScaler to scale the feature values

Step 7: Initialize K-Nearest Neighbours and Logistic Regression classifiers

Step 8: Train both models using the training dataset

Step 9: Predict outputs using the test dataset

Step 10: Evaluate both models using accuracy score

Step 11: Compare accuracy of both models

Step 12: Display results

Step 13: Stop

Program :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier()
lr = LogisticRegression(max_iter=1000)

knn.fit(X_train, y_train)
lr.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)
y_pred_lr = lr.predict(X_test)

acc_knn = accuracy_score(y_test, y_pred_knn)
acc_lr = accuracy_score(y_test, y_pred_lr)

print("KNN Accuracy:", acc_knn)
print("Logistic Regression Accuracy:", acc_lr)

if acc_knn > acc_lr:
    print("KNN performs better")
elif acc_lr > acc_knn:
    print("Logistic Regression performs better")
else:
    print("Both models perform equally")
```

<u>output :</u>

```
KNN Accuracy: 1.0
Logistic Regression Accuracy: 1.0
Both models perform equally
```

b)

algorithm:

Step 1: Start

Step 2: Import required libraries

Step 3: Load the Iris dataset

Step 4: Convert dataset into a DataFrame with feature names

Step 5: Add target variable to the DataFrame

Step 6: Map numeric class labels to species names

Step 7: Create a pairplot using Seaborn with hue as species

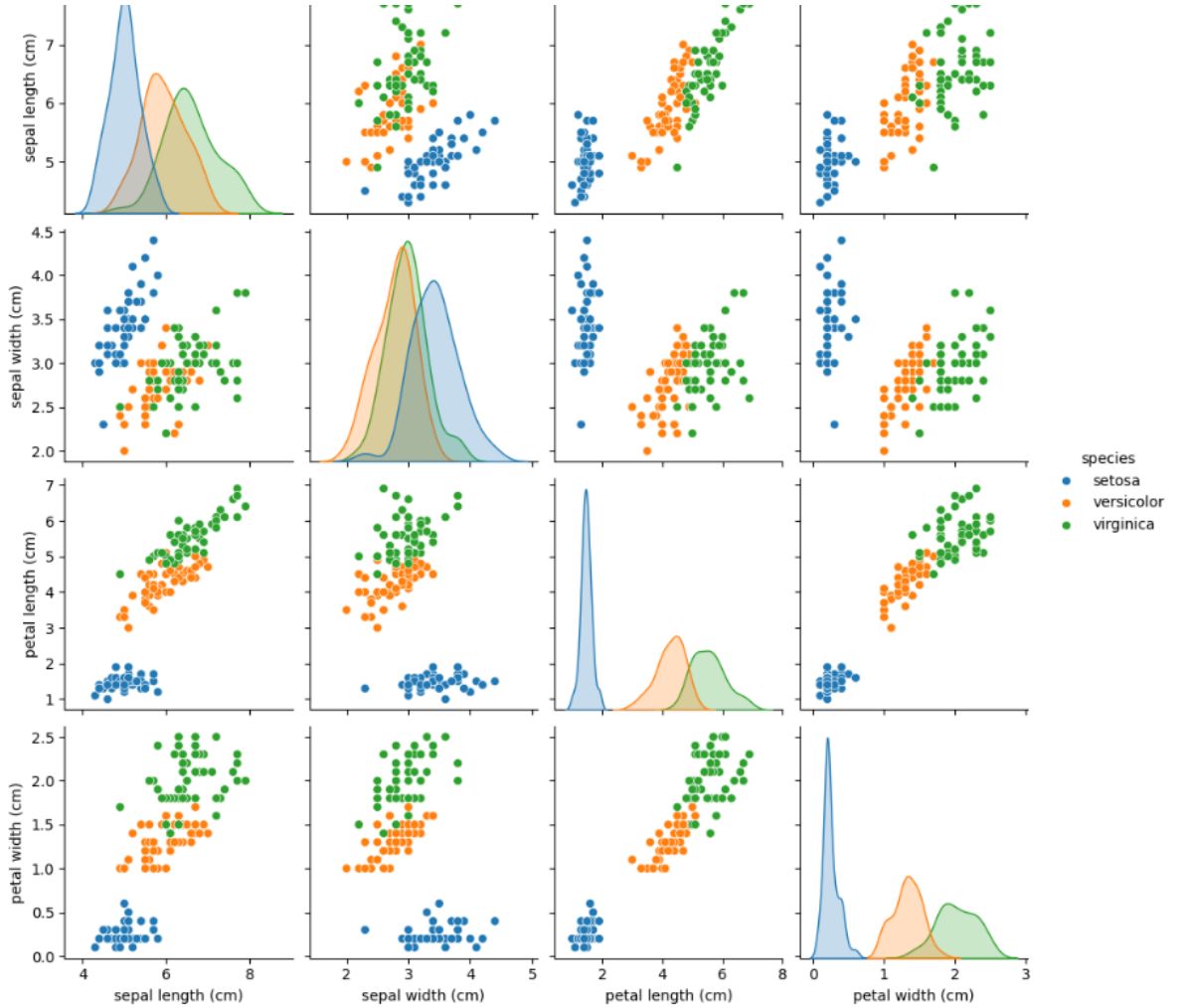
Step 8: Display the plot

Step 9: Stop

program:

```
import seaborn as sns
import pandas as pd
from sklearn.datasets import load_iris

data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['species'] = data.target
df['species'] = df['species'].map({0: 'setosa', 1: 'versicolor', 2: 'virginica'})

sns.pairplot(df, hue='species')
```

output :



Q7)

a) Load the Iris dataset from sklearn.datasets and split it into training (80%) and test (20%) sets. Apply MinMaxScaler for feature scaling, then train a Decision Tree and an SVM classifier on the training set. Predict on the test set using each model and compare their performance using accuracy score.

b) Using Matplotlib, visualise the Decision Tree structure trained on the Iris dataset up to a depth of 3 using plot_tree, labelling the feature names and class names.

a)

algorithm :

Step 1: Start

Step 2: Import required libraries

Step 3: Load the Iris dataset

Step 4: Separate features (X) and target variable (y)

Step 5: Split the dataset into training (80%) and testing (20%) sets

Step 6: Apply MinMaxScaler to scale the feature values

Step 7: Initialize Decision Tree and SVM classifiers

Step 8: Train both models using the training dataset

Step 9: Predict outputs using the test dataset

Step 10: Evaluate both models using accuracy score

Step 11: Compare accuracy of both models

Step 12: Display results

Step 13: Stop

Program :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

dt = DecisionTreeClassifier()
svm = SVC()

dt.fit(X_train, y_train)
svm.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
y_pred_svm = svm.predict(X_test)

acc_dt = accuracy_score(y_test, y_pred_dt)
acc_svm = accuracy_score(y_test, y_pred_svm)

print("Decision Tree Accuracy:", acc_dt)
print("SVM Accuracy:", acc_svm)

if acc_dt > acc_svm:
    print("Decision Tree performs better")
elif acc_svm > acc_dt:
    print("SVM performs better")
else:
    print("Both models perform equally")
```

output:

Decision Tree Accuracy: 1.0

SVM Accuracy: 1.0

Both models perform equally

b)

algorithm :

Step 1: Start

Step 2: Import required libraries

Step 3: Load the Iris dataset

Step 4: Separate features (X) and target variable (y)

Step 5: Split the dataset into training and testing sets

Step 6: Train a Decision Tree classifier with maximum depth set to 3

Step 7: Use Matplotlib to create a figure

Step 8: Visualise the trained Decision Tree using plot_tree

Step 9: Provide feature names and class names for labeling

Step 10: Display the plot

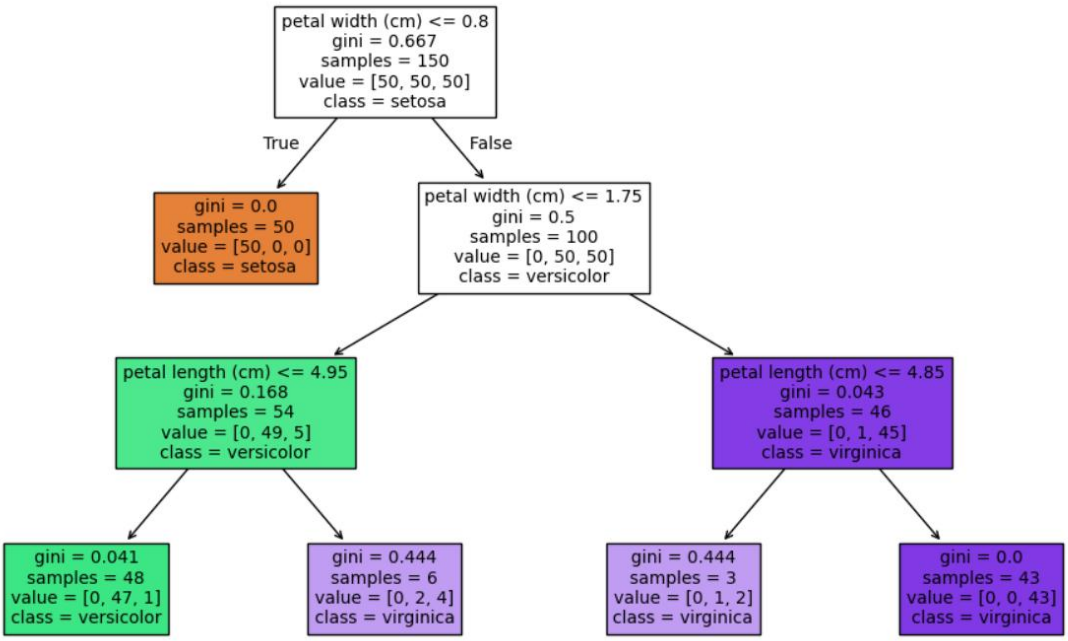
Step 11: Stop

Program :

```
import matplotlib.pyplot as plt

from sklearn.datasets import load_iris

from sklearn.tree import DecisionTreeClassifier, plot_tree


data = load_iris()

X = data.data

y = data.target
```

```
dt = DecisionTreeClassifier(max_depth=3)

dt.fit(X, y)


plt.figure(figsize=(10,6))

plot_tree(dt, feature_names=data.feature_names, class_names=data.target_names,
filled=True)

plt.tight_layout()

plt.show()

plt.close()
```

output:



Q8)

a) Load the Iris dataset from sklearn.datasets and split it into training (80%) and test (20%) sets. Apply StandardScaler for feature scaling, then train a Naive Bayes and a Random Forest classifier on the training set. Predict on the test set using each model and compare their performance using accuracy score.

b) Using Seaborn, plot a heatmap of the confusion matrix for each classifier, clearly labelling the Iris species on both axes.

a)

algorithm :

Step 1: Start

Step 2: Import required libraries

Step 3: Load the Iris dataset

Step 4: Separate features (X) and target variable (y)

Step 5: Split the dataset into training (80%) and testing (20%) sets

Step 6: Apply StandardScaler to scale the feature values

Step 7: Initialize Naive Bayes and Random Forest classifiers

Step 8: Train both models using the training dataset

Step 9: Predict outputs using the test dataset

Step 10: Evaluate both models using accuracy score

Step 11: Compare accuracy of both models

Step 12: Display results

Step 13: Stop

Program :

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.naive_bayes import GaussianNB

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score


data = load_iris()

X = data.data

y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

nb = GaussianNB()
rf = RandomForestClassifier()

nb.fit(X_train, y_train)
rf.fit(X_train, y_train)

y_pred_nb = nb.predict(X_test)
y_pred_rf = rf.predict(X_test)

acc_nb = accuracy_score(y_test, y_pred_nb)
acc_rf = accuracy_score(y_test, y_pred_rf)

print("Naive Bayes Accuracy:", acc_nb)
print("Random Forest Accuracy:", acc_rf)

if acc_nb > acc_rf:
    print("Naive Bayes performs better")
elif acc_rf > acc_nb:
    print("Random Forest performs better")
else:
```

```
print("Both models perform equally")
```

output :

Naive Bayes Accuracy: 1.0

Random Forest Accuracy: 1.0

Both models perform equally

b)

algorithm :

Step 1: Start

Step 2: Import required libraries

Step 3: Load the Iris dataset and split into training and testing sets

Step 4: Apply StandardScaler to the feature data

Step 5: Train Naive Bayes and Random Forest classifiers

Step 6: Predict class labels on the test dataset for both models

Step 7: Compute confusion matrices for both classifiers

Step 8: Define class labels using Iris species names

Step 9: Plot heatmap for Naive Bayes confusion matrix with proper labels

Step 10: Plot heatmap for Random Forest confusion matrix with proper labels

Step 11: Display both heatmaps

Step 12: Stop

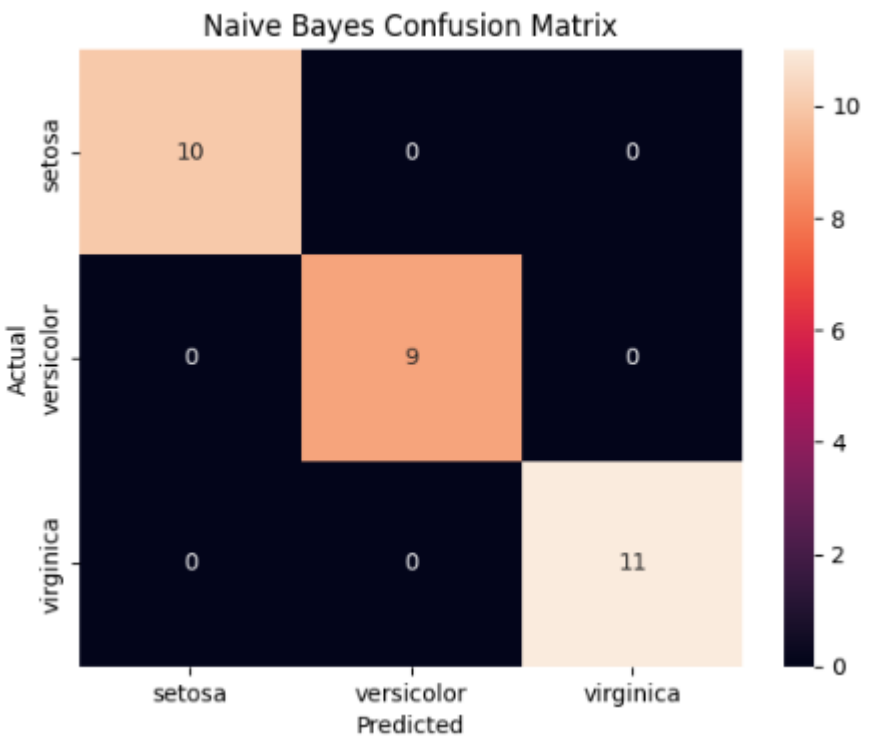
Program :

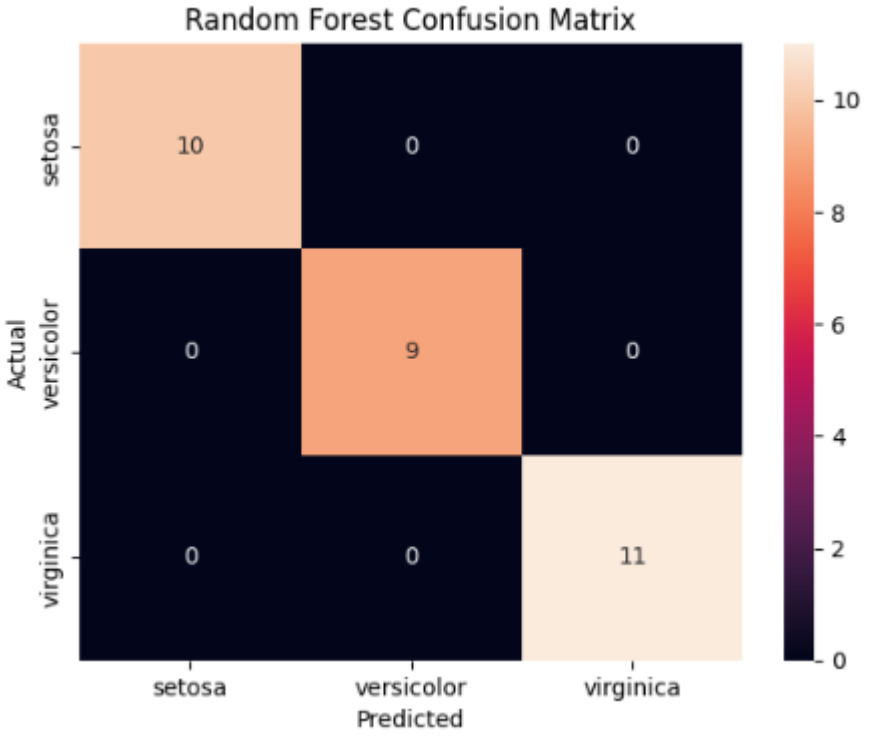
```
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

nb = GaussianNB()
rf = RandomForestClassifier()

nb.fit(X_train, y_train)
rf.fit(X_train, y_train)

y_pred_nb = nb.predict(X_test)
y_pred_rf = rf.predict(X_test)

labels = data.target_names

cm_nb = confusion_matrix(y_test, y_pred_nb)
```

```
plt.figure()
sns.heatmap(cm_nb, annot=True, fmt='d', xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Naive Bayes Confusion Matrix")
plt.show()

cm_rf = confusion_matrix(y_test, y_pred_rf)
plt.figure()
sns.heatmap(cm_rf, annot=True, fmt='d', xticklabels=labels, yticklabels=labels)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Random Forest Confusion Matrix")
plt.show()
```

output :


Naive Bayes Confusion Matrix
setosa
versicolor
virginica
Actual
10
0
0
0
9
0
0
0
11
setosa
versicolor
virginica
Predicted
10
8
6
4
2
0


Random Forest Confusion Matrix
setosa
versicolor
virginica
Actual
10
0
0
0
9
0
0
0
11
setosa
versicolor
virginica
Predicted
10
8
6
4
2
0

Q9)

a) Load the Iris dataset from sklearn.datasets and split it into training (80%) and test (20%) sets. Apply MinMaxScaler for feature scaling, then train K-Nearest Neighbours and Naive Bayes classifiers on the training set. Predict on the test set using each model and compare their performance using accuracy score.
b) Using Matplotlib, plot a bar chart comparing the accuracy scores of both classifiers, and a separate line graph showing the effect of varying K values (1 to 10) on KNN accuracy.

a)

algorithm :

Step 1: Start

Step 2: Import required libraries

Step 3: Load the Iris dataset

Step 4: Separate features (X) and target variable (y)

Step 5: Split the dataset into training (80%) and testing (20%) sets

Step 6: Apply MinMaxScaler to scale the feature values

Step 7: Initialize K-Nearest Neighbours and Naive Bayes classifiers

Step 8: Train both models using the training dataset

Step 9: Predict outputs using the test dataset

Step 10: Evaluate both models using accuracy score

Step 11: Compare accuracy of both models

Step 12: Display results

Step 13: Stop

Program :

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import MinMaxScaler

from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier()
nb = GaussianNB()

knn.fit(X_train, y_train)
nb.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)
y_pred_nb = nb.predict(X_test)

acc_knn = accuracy_score(y_test, y_pred_knn)
acc_nb = accuracy_score(y_test, y_pred_nb)

print("KNN Accuracy:", acc_knn)
```

```
print("Naive Bayes Accuracy:", acc_nb)

if acc_knn > acc_nb:
    print("KNN performs better")
elif acc_nb > acc_knn:
    print("Naive Bayes performs better")
else:
    print("Both models perform equally")
```

output :

KNN Accuracy: 1.0

Naive Bayes Accuracy: 1.0

Both models perform equally

b)

algorithm :

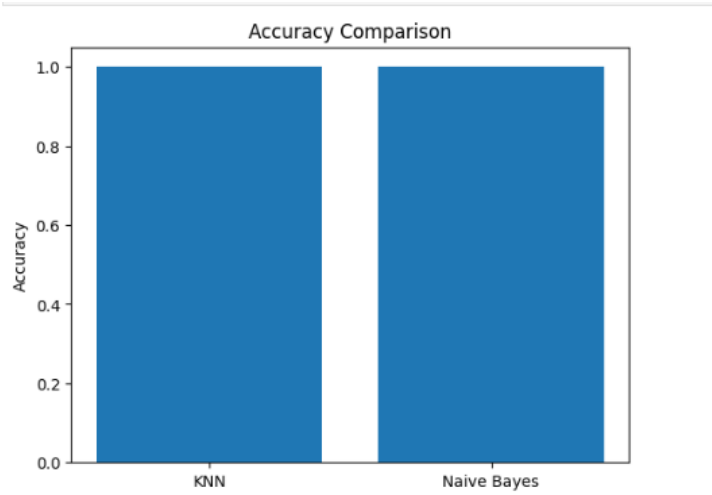
- Load Iris dataset.
- Split dataset into training (80%) and testing (20%).
- Apply MinMaxScaler to scale features.
- Train KNN classifier on training data.
- Train Naive Bayes classifier on training data.
- Predict test data using both models.
- Calculate accuracy scores for both classifiers.
- Plot bar chart comparing accuracies.
- For K values from 1 to 10, train KNN and record accuracy.
- Plot line graph of K vs accuracy.

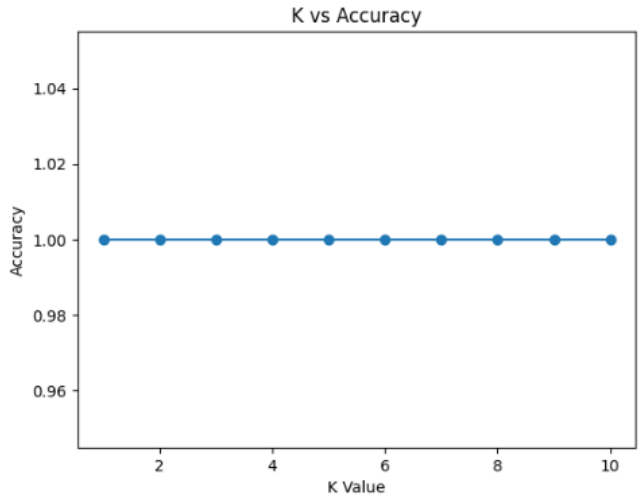
Program :

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=3)
nb = GaussianNB()

knn.fit(X_train, y_train)
nb.fit(X_train, y_train)

y_pred_knn = knn.predict(X_test)
y_pred_nb = nb.predict(X_test)
```

```
acc_knn = accuracy_score(y_test, y_pred_knn)
acc_nb = accuracy_score(y_test, y_pred_nb)

plt.bar(['KNN', 'Naive Bayes'], [acc_knn, acc_nb])
plt.title('Accuracy Comparison')
plt.ylabel('Accuracy')
plt.show()

k_values = range(1, 11)
accuracies = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    accuracies.append(accuracy_score(y_test, pred))

plt.plot(k_values, accuracies, marker='o')
plt.title('K vs Accuracy')
plt.xlabel('K Value')
plt.ylabel('Accuracy')
plt.show()
```

output :





Q10)

a) Load the Iris dataset from sklearn.datasets and split it into training (80%) and test (20%) sets. Apply StandardScaler for feature scaling, then train Logistic Regression and Random Forest classifiers on the training set. Predict on the test set using each model and compare their performance using accuracy score.
b) Using Seaborn, plot a boxplot showing the distribution of each of the four features across the three Iris species, and a heatmap of the feature correlation matrix.

a)

algorithm :

- Load Iris dataset.
- Split dataset into training (80%) and testing (20%).

- Apply StandardScaler to scale features.
- Train Logistic Regression model on training data.
- Train Random Forest model on training data.
- Predict test data using both models.
- Calculate accuracy scores for both classifiers.
- Compare the accuracy results.

Program :

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

lr = LogisticRegression(max_iter=200)
rf = RandomForestClassifier(random_state=42)
```

```
lr.fit(X_train, y_train)
rf.fit(X_train, y_train)

y_pred_lr = lr.predict(X_test)
y_pred_rf = rf.predict(X_test)

acc_lr = accuracy_score(y_test, y_pred_lr)
acc_rf = accuracy_score(y_test, y_pred_rf)

print("Logistic Regression Accuracy:", acc_lr)
print("Random Forest Accuracy:", acc_rf)
```

output :

Logistic Regression Accuracy: 1.0

Random Forest Accuracy: 1.0

b)

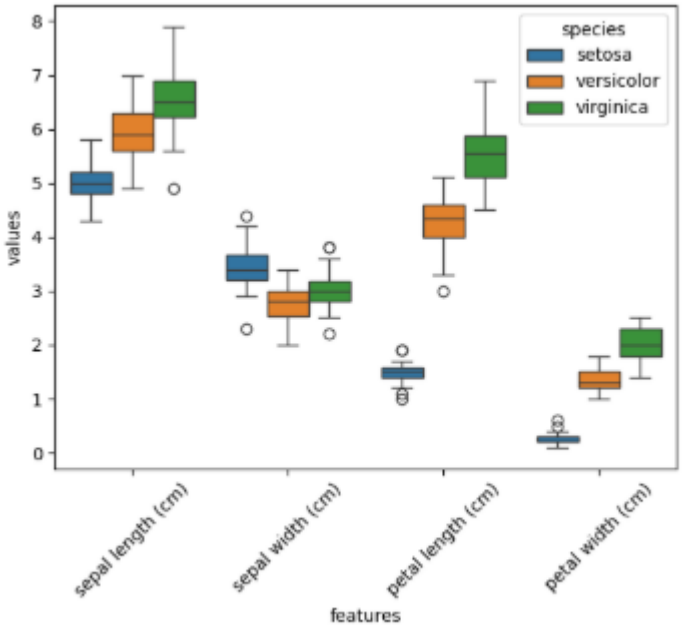
algorithm :

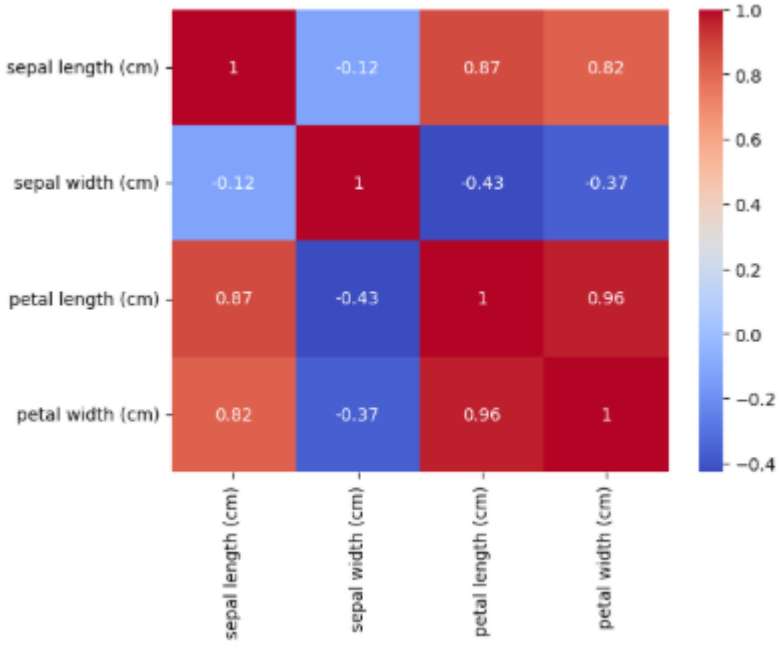
1. Load Iris dataset.
2. Convert dataset into a DataFrame with feature names.
3. Add target labels and map them to species names.
4. Reshape data to long format for boxplot.
5. Plot boxplot of features vs species using Seaborn.
6. Compute correlation matrix of features.
7. Plot heatmap of correlation matrix.

Program :

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

data = load_iris()
df = pd.DataFrame(data.data, columns=data.feature_names)
df['species'] = data.target
df['species'] = df['species'].map({i: name for i, name in enumerate(data.target_names)})

df_melt = pd.melt(df, id_vars='species', var_name='features', value_name='values')

sns.boxplot(x='features', y='values', hue='species', data=df_melt)
plt.xticks(rotation=45)
plt.show()

corr = df.drop('species', axis=1).corr()

sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.show()
```

Output :


species
setosa
versicolor
virginica
values
features
sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)


sepal length (cm)
sepal width (cm)
petal length (cm)
petal width (cm)
1
-0.12
0.87
0.82
-0.12
1
-0.43
-0.37
0.87
-0.43
1
0.96
0.82
-0.37
0.96
1